

Data Preprocessing

Initial steps before building a machine learning model

Nataly Jaya Salazar

Summer of 2026

Contents

1	Introduction	3
1.1	What is Data Preprocessing	3
1.2	Dataset	3
2	Environment Setup and Libraries	3
2.1	Creating an Environment	3
2.2	Libraries	4
3	Reading the Dataset	4
4	Sanity Check of Data	5
4.1	Dimensions and Data Types	5
4.2	Missing Values	5
4.3	Duplicates and Garbage Values	6
5	Exploratory Data Analysis (EDA)	6
5.1	Descriptive Statistics	6
5.2	Distributions	7
5.3	Outlier Inspection	7
5.4	Relationships with the Target Variable	7
5.5	Correlation and Multicollinearity	8
6	Missing Value Treatments	8
6.1	Choosing a Strategy	8
6.2	Simple Imputation with the Median	9
6.3	KNN Imputation for the Remaining Columns	9
7	Outliers Treatment	9
7.1	The IQR (Whisker) Method	9
7.2	Capping the Outliers	10
8	Duplicates and Garbage Value Treatment	10

Data preprocessing

Initial steps before building a machine learning model Nataly Jaya Salazar

9 Encoding of Data **10**

10 Conclusion **11**

1 Introduction

1.1 What is Data Preprocessing

Data preprocessing consists of cleaning, transforming, and organizing raw data so that it is suitable for building a machine learning model or performing any other kind of data analysis. Real-world data is almost never “model-ready”: it contains missing values, inconsistent formats, outliers, and duplicated records. Skipping this step usually leads to a model that learns from noise instead of from the real signal.

1.2 Dataset

The dataset used throughout this report is the *Life Expectancy (WHO)* dataset, available on Kaggle¹. Before applying any cleaning procedure, it is important to first understand how the data is (dis)organized, so that every transformation applied afterward is justified rather than arbitrary. Figure ?? shows an example of the first rows of the raw dataset.

2 Environment Setup and Libraries

2.1 Creating an Environment

Before writing any code, it is good practice to isolate the project in its own Python environment². This keeps the libraries used for this project separate from the rest of the system and avoids version conflicts.

```
python3 -m venv env          # Creates the virtual env
source env/bin/activate     # Activates the env
```

¹<https://www.kaggle.com/datasets/kumarajarshi/life-expectancy-who>

²Make sure Python is installed on your machine. See <https://comp-think.github.io/laboratory/site/chapter/01/> for setup instructions.

Data preprocessing

Initial steps before building a machine learning model Nataly Jaya Salazar

The environment is then registered as a Jupyter kernel so it can be selected from a notebook:

```
pip install ipykernel
python3 -m ipykernel install --user --name=ml_env --display-name "ML
    Kernel"
```

2.2 Libraries

A library is imported in Python using the structure `import library_name as abbreviation`.

For this project, four libraries are used:

- **Pandas**: manipulation and analysis of tabular data.
- **NumPy**: efficient management of multi-dimensional arrays and numerical operations.
- **Seaborn**: statistical data visualization, built on top of Matplotlib.
- **Matplotlib.pyplot**: creation of static, animated, and interactive visualizations.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

3 Reading the Dataset

The CSV file is loaded into a **DataFrame**, the main structure Pandas uses to store tabular data, so that it can be manipulated in Python. Loading the data is the entry point of the whole pipeline: every cleaning and analysis step described in the following sections is applied to this object.

```
df = pd.read_csv("Life Expectancy Data.csv")
```

Before doing anything else, a quick look at the beginning and the end of the dataset gives a first impression of the columns available (*Country*, *Year*, *Status*, *Life expectancy*, *Adult Mortality*, etc.), how they are named, and whether the values look reasonable, before trusting the data.

```
df.head()  # first 5 rows
df.tail()  # last 5 rows
```

4 Sanity Check of Data

4.1 Dimensions and Data Types

The shape of the dataset and the type of each column are checked first:

```
df.shape    # dimensions of the matrix
df.info()   # non-null count + dtype per column
```

The dataset has **2938 rows and 22 columns**. `df.info()` reports, for every column, how many non-null values it has and its data type (`object`, `int64`, `float64`). This is the fastest way to spot columns with missing data and columns with an incorrect type (e.g. a numeric column stored as text).

4.2 Missing Values

```
df.isnull().sum()                # count of missing values
df.isnull().sum() / df.shape[0] * 100  # % of missing values
```

For example, *Hepatitis B* has 553 missing values ($\approx 18.8\%$), *Population* has 652 missing values ($\approx 22.2\%$), and *Alcohol* has 194 missing values ($\approx 6.6\%$). Knowing how many, and

what percentage, of values are missing per column determines whether rows can safely be dropped or whether an imputation strategy is required (Section 6).

4.3 Duplicates and Garbage Values

```
df.duplicated().sum() # counts fully duplicated rows

for i in df.select_dtypes(include="object").columns:
    print(df[i].value_counts())
```

In this dataset, `duplicated().sum()` returns **0**: there are no fully duplicated rows. `value_counts()` on the categorical columns (*Country*, *Status*) confirms 193 valid countries and only two consistent categories (*Developing*, *Developed*), with no typos or inconsistent labels. Duplicated rows would bias a model by repeating the same information, and inconsistent category labels (e.g. “USA” vs. “United States”) would incorrectly be treated as different groups.

5 Exploratory Data Analysis (EDA)

5.1 Descriptive Statistics

```
df.describe().T # numeric summary
df.describe(include="object") # categorical summary
```

`describe()` returns the count, mean, standard deviation, minimum, maximum, and quartiles for every numeric column (`.T` transposes the table so it is easier to read column by column). These summary statistics make it possible to quickly spot suspicious values and give a sense of scale before choosing a normalization strategy.

5.2 Distributions

```
for i in df.select_dtypes(include="number").columns:
    sns.histplot(data=df, x=i)
    plt.show()
```

Histograms show the shape of each variable's distribution (normal, skewed, bimodal), which helps decide whether a transformation, or a particular imputation method (mean vs. median), is appropriate.

5.3 Outlier Inspection

```
for i in df.select_dtypes(include="number").columns:
    sns.boxplot(data=df, x=i)
    plt.show()
```

Boxplots visualize the median, the interquartile range, and the points that fall outside the whiskers, giving a first visual clue about which columns (e.g. *GDP*, *Population*) contain outliers to be treated later (Section 7).

5.4 Relationships with the Target Variable

```
for i in ['Year', 'Adult Mortality', 'Alcohol', 'GDP', 'Schooling',
...]:
    sns.scatterplot(data=df, x=i, y='Life expectancy ')
    plt.show()
```

Plotting every numeric feature against the target variable (*Life expectancy*) shows, before modelling, which variables have a linear, non-linear, or negligible relationship with the quantity to be predicted.

5.5 Correlation and Multicollinearity

```
s = df.select_dtypes(include="number").corr()  
plt.figure(figsize=(15, 15))  
sns.heatmap(s, annot=True)
```

The correlation heatmap reveals which variables correlate with *Life expectancy*, and which independent variables correlate strongly with each other. This second point, known as multicollinearity, is a problem for several ML models, such as linear regression.

6 Missing Value Treatments

6.1 Choosing a Strategy

Once the location of every missing value is known, a strategy to fill them in must be chosen.

Common approaches include:

- **Mean** imputation, for symmetric, non-skewed distributions.
- **Median** imputation, more robust when the distribution is skewed or contains outliers.
- **Mode** imputation, for categorical variables.
- **KNNImputer**, which estimates a missing value from the values of its nearest neighbours (similar rows).

There is no single correct method: the right choice depends on the distribution of each variable and on how much information can be afforded to be approximated.

6.2 Simple Imputation with the Median

```
for i in [" BMI ", "Polio", "Income composition of resources"]:  
    df[i].fillna(df[i].median(), inplace=True)  
    # inplace=True overwrites the column instead of creating a copy
```

These three columns showed skewed distributions in the histograms of Section 5.2, so the median was chosen because it is less sensitive to extreme values than the mean.

6.3 KNN Imputation for the Remaining Columns

```
from sklearn.impute import KNNImputer  
impute = KNNImputer()  
  
for i in df.select_dtypes(include="number").columns:  
    df[i] = impute.fit_transform(df[[i]])
```

After running this step, `df.isnull().sum()` returns 0 missing values across every column. Instead of filling every missing value with the same constant, `KNNImputer` estimates a value based on similar rows, which usually preserves more of the original structure of the data [geeksforgeeks].

7 Outliers Treatment

7.1 The IQR (Whisker) Method

The lower and upper whiskers defined in Equations (2) and (3) are computed for each numeric column, for example:

```
def whisker(col):  
    q1, q3 = np.percentile(col, [25, 75])  
    iqr = q3 - q1
```

```
lw = q1 - 1.5 * iqr
uw = q3 + 1.5 * iqr
return lw, uw

wisker(df['GDP']) # -> (-9773.52, 17837.17)
```

Any value below the lower whisker or above the upper whisker is considered a statistical outlier, following the same logic a boxplot uses to draw its whiskers.

7.2 Capping the Outliers

```
for i in ['GDP', 'Total expenditure', 'thinness 5-9 years']:
    lw, uw = wisker(df[i])
    df[i] = np.where(df[i] < lw, lw, df[i])
    df[i] = np.where(df[i] > uw, uw, df[i])
```

Rather than deleting the affected rows, which would waste valid information from the other columns, extreme values are capped (also called *winsorizing*) to the nearest reasonable bound.

8 Duplicates and Garbage Value Treatment

```
df.drop_duplicates()
```

In this dataset, `duplicated().sum()` was already 0 (Section 4.3), so this step confirms there is nothing to remove. It is still run as good practice on any new dataset: even when a quick check shows no duplicates, explicitly calling `drop_duplicates()` documents that the check was performed and protects the pipeline if the raw data changes later.

9 Encoding of Data

```
pd.get_dummies(data=df, columns=["Country", "Status"], drop_first=True)
```

This transforms each category of *Country* and *Status* into its own binary (0/1) column, a technique known as one-hot encoding. The option `drop_first=True` removes the first category of each variable to avoid redundant columns. Machine learning models work with numbers, not text, so encoding is what allows a categorical variable such as *Status* (“Developing”/“Developed”) to be used as a numeric input.

10 Conclusion

Following the pipeline above, missing values, outliers, duplicates, and categorical variables in the *Life Expectancy (WHO)* dataset were identified and corrected. The dataset is now numeric, complete, and free of duplicated or extreme values, and is therefore ready for the feature-scaling and modelling stages of the machine learning workflow.